Course Project

- Anything related to computer vision

- Some of the previous projects
  - Vide game AI (vision + reinforcement learning)
  - Detecting damaged buildings in satellite images
  - Recognition of histopathology images
  - Novel view synthesis with NeRF (3D vision)
  - Evaluation of Transformer models for VQA

Course Project

- Anything related to computer vision
- Team-based (2-3).
  - 3 is our preferred number.
  - Teams of 4 needs instructor approval.
  - Individual projects are *strongly discouraged.*
- Find your teammates and discuss the project!
- Milestones
  - Proposal (single page)
  - Mid-term report (single page)
  - Final presentation & report (4 pages)

Project Proposal

- Due Oct 12th (single page document)
- Please describe what you plan to do, and why it is interesting (problem and motivation)
- It will be great if you have a tentative solution to the problem … it is OK that you don’t have a concrete plan
- We will provide some feedback based on your proposal

Recurrent Neural Networks and Transformers (Part I)

Many slides from Lana Lazebnik, Arun Mallya

Let us look at problems with sequences as inputs or outputs

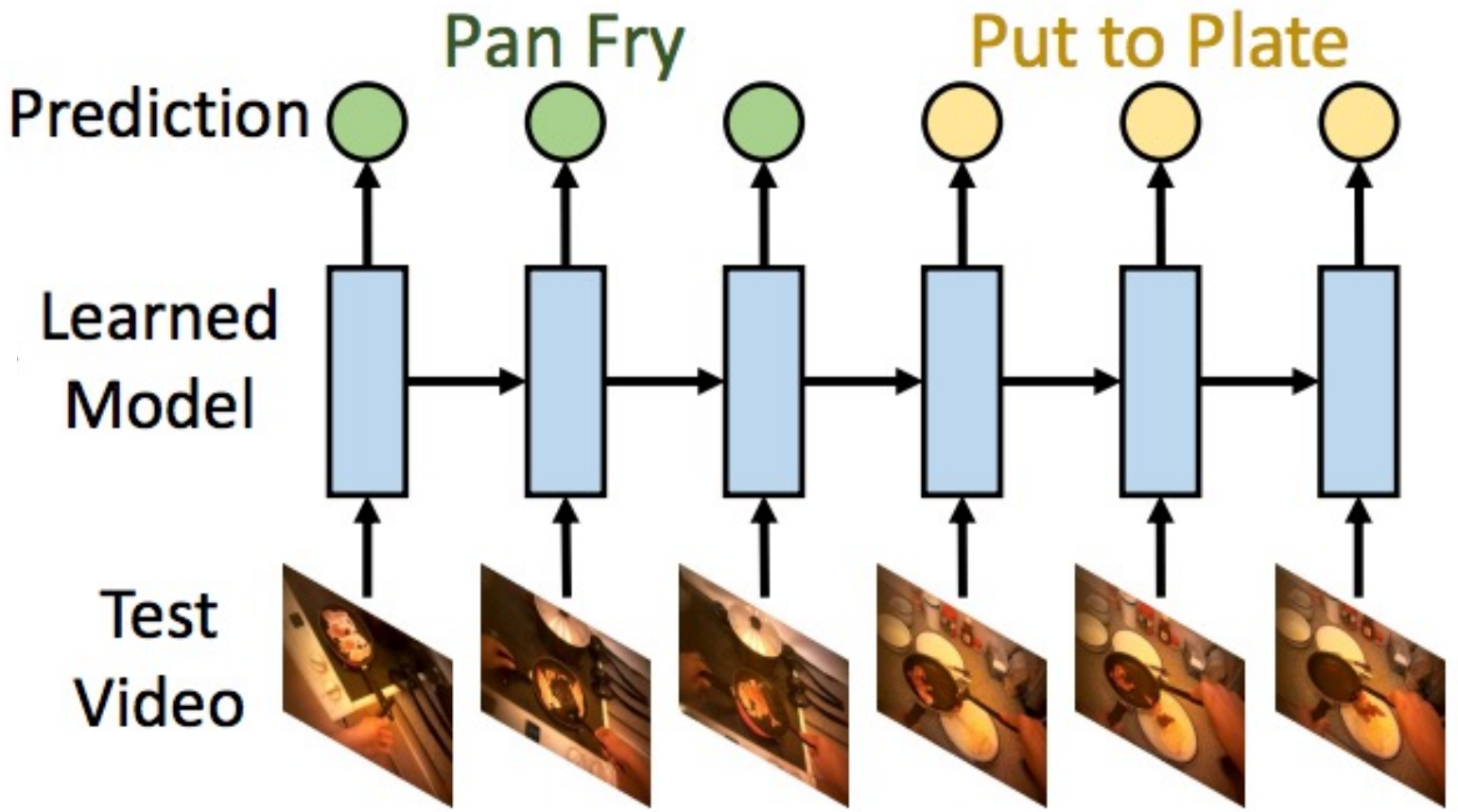
Temporal Action Segmentation

- Given a video, annotate each frame with an action / event label



[Huang, Fei-Fei, Niebles]

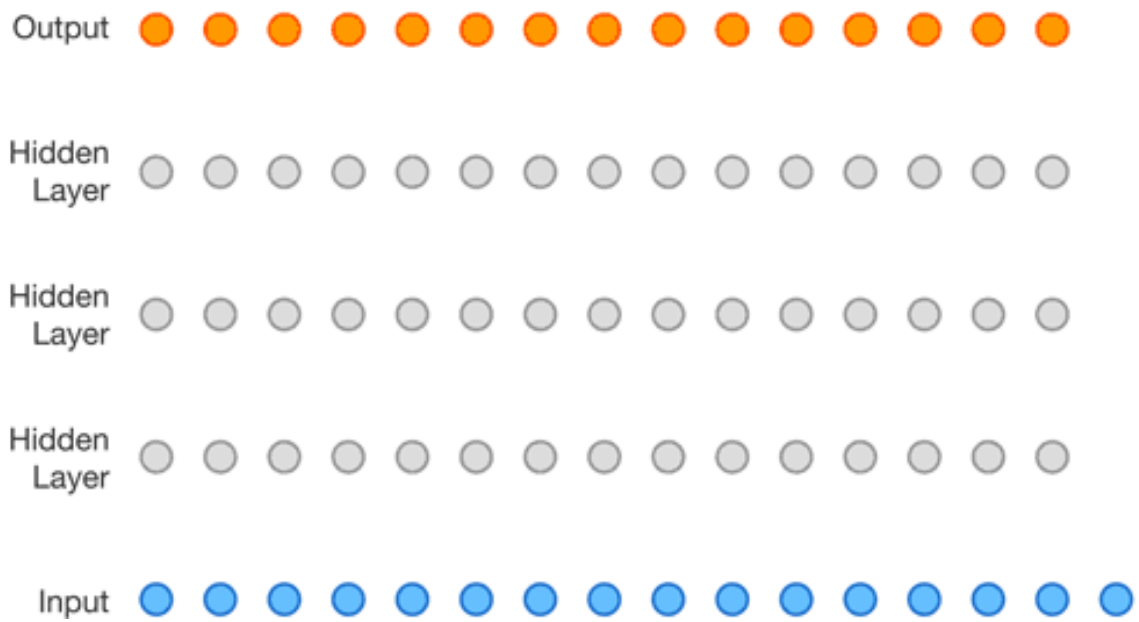
Temporal Action Segmentation



[Huang, Fei-Fei, Niebles]

Time Series Prediction using 1D ConvNets

- 1D feed-forward convolutional networks
  - Can be generative using autoregressive decoding
  - Once the model architecture is defined, **the output size is fixed or determined by the input size**



WaveNet: A Generative Model for Raw Audio, DeepMind

Sequential Prediction Tasks

- ConvNets:
  - "fixed" mapping on regular grids *(given a model architecture, the output size is fixed or determined by the input size)*
  - Limited memory about the history (small receptive field)

- *What if the input and/or output is a variable-length sequence?*

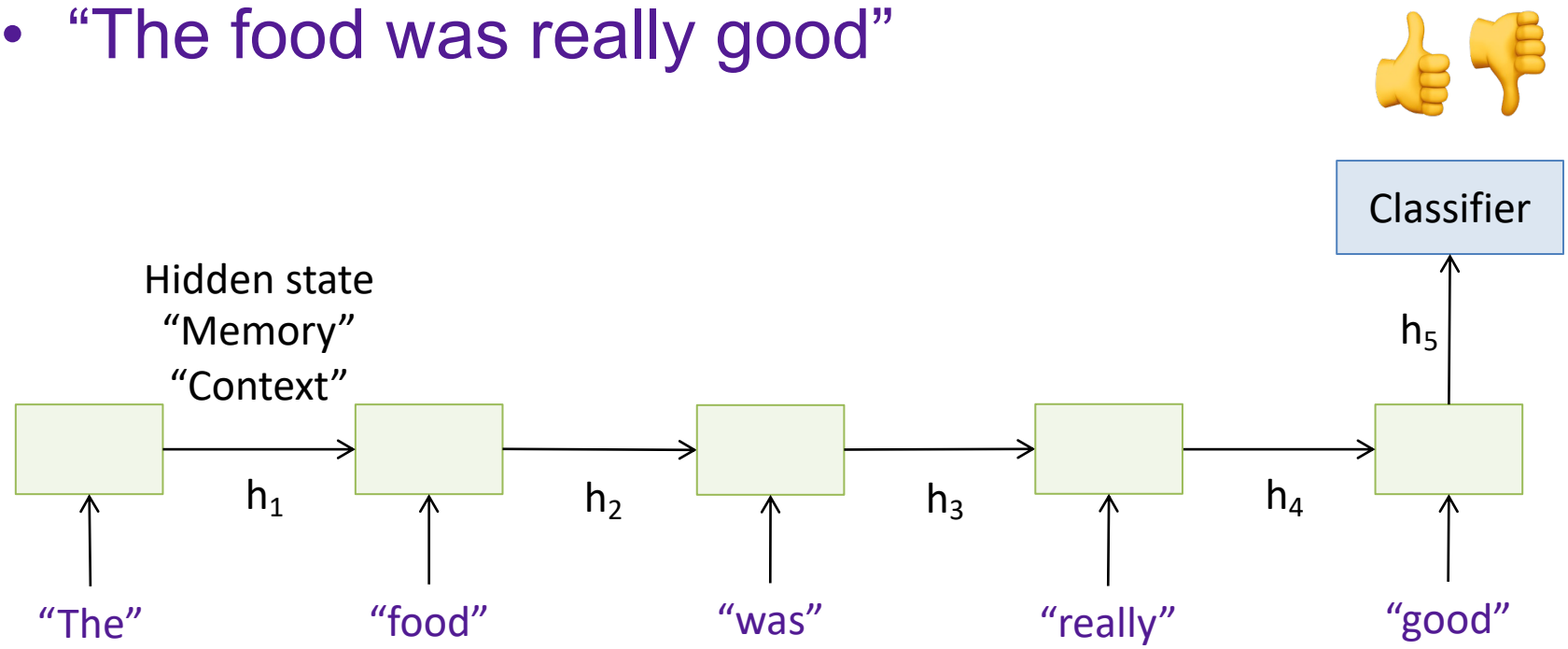
Text Classification

- **Sentiment classification:** classify a restaurant or movie or product review as positive or negative

  - "The food was really good"
  - "The vacuum cleaner broke within two weeks"
  - "The movie had slow parts, but overall was worth watching"

Sentiment Classification

- "The food was really good"

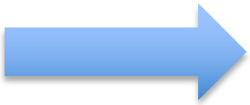


Recurrent Neural Network (RNN)

Image Caption Generation

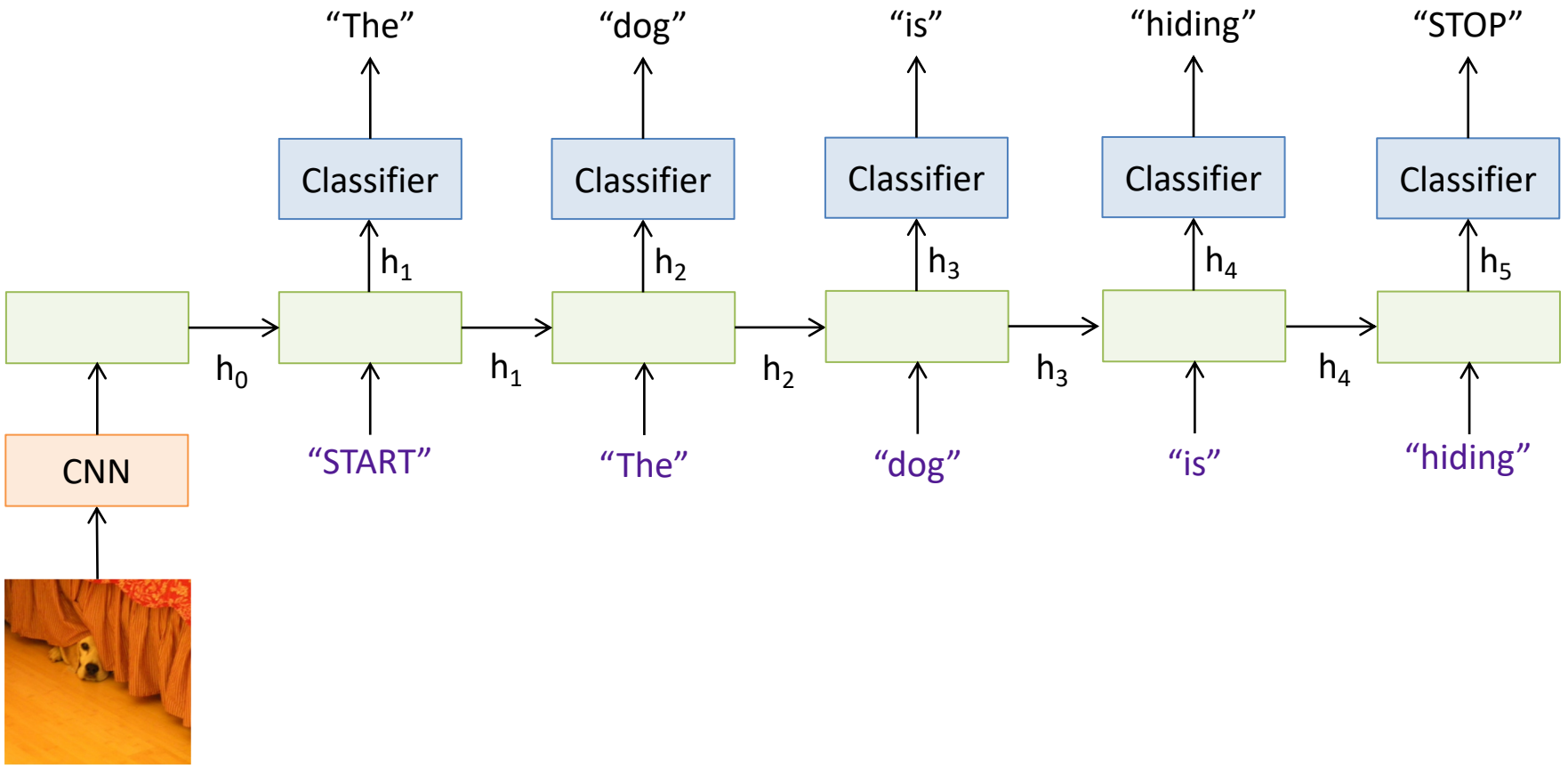
- **Image captioning**: given an image, produce a sentence describing its contents





"The dog is hiding"

Image Caption Generation



AI Chatbot

Y

How does language language model work? Can you describe brieft?

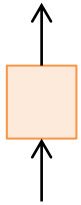
Certainly! At a high level, a language model like ChatGPT (based on OpenAI's GPT-4 or similar architectures) is a type of machine learning model designed to generate and understand human language. Here's a brief overview:

1. **Training Data**: Language models are trained on vast amounts of text data. This data helps the model learn the structure and nuances of the language.
2. **Architecture**: The architecture of the model, often based on neural networks, dictates how it processes and learns from this data. GPT-4, like its predecessors, uses a type of architecture called a Transformer.
3. **Learning Patterns**: During training, the model adjusts its internal parameters to better predict the next word in a sequence. For example, given the input "The sky is...", it might learn to predict "blue" as a likely next word.
4. **Contextual Understanding**: Unlike earlier language models, modern models like GPT-4 can consider the context of a whole passage of text. So, the prediction for the next word or how

https://chat.openai.com/

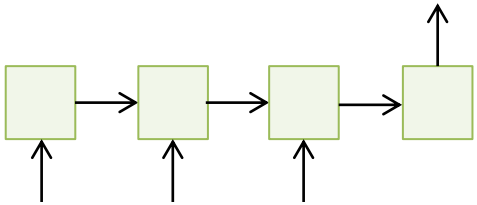
Summary: Input-output Scenarios

Single - Single



Feed-forward Network
(ConvNets)

Multiple - Single



Sentiment Classification
(ConvNets?)

Single - Multiple

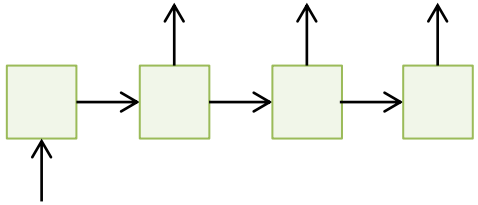
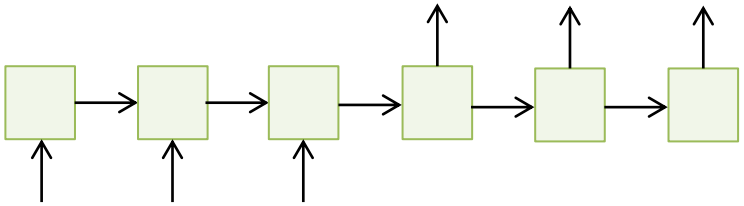


Image Captioning
(ConvNets?)

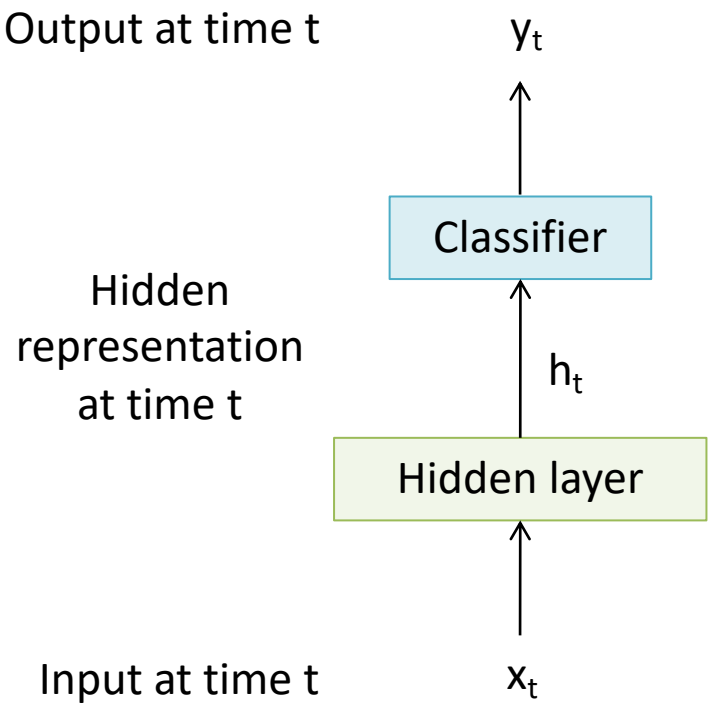
Multiple - Multiple



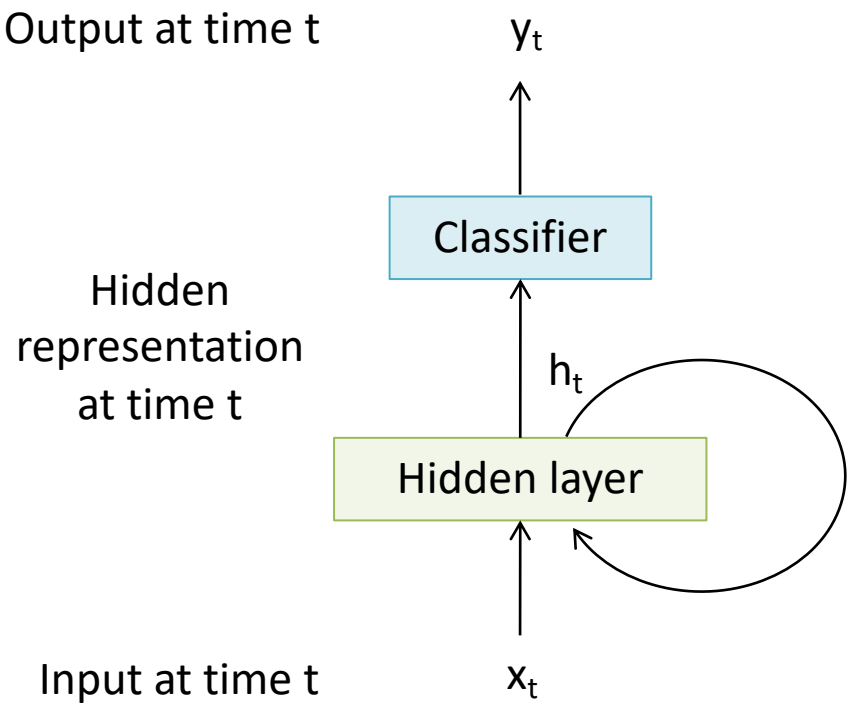
Translation / Chatbot
(ConvNets?)

Recurrent Neural Networks

Neural Network (NN)



Recurrent Neural Network (RNN)

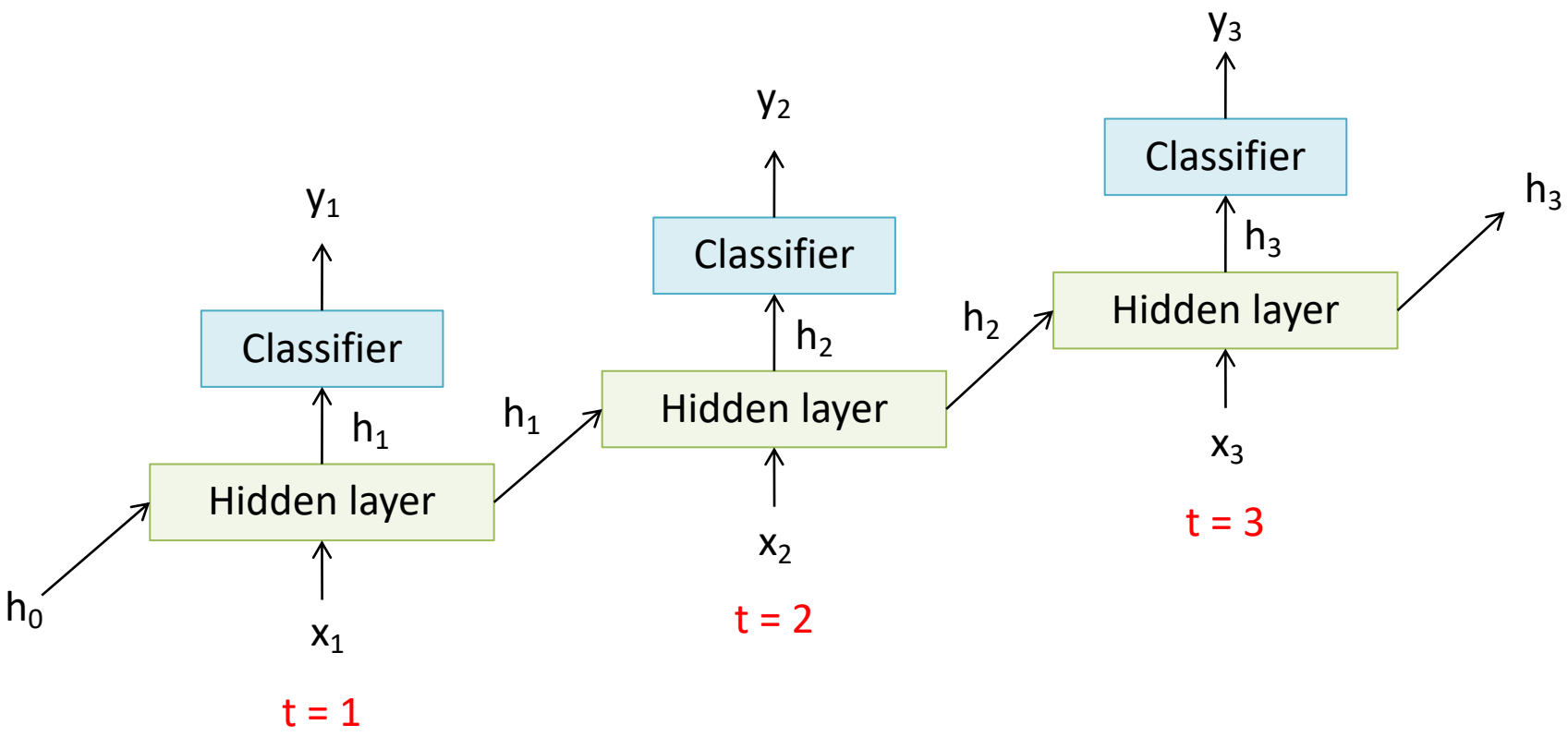


Recurrence:

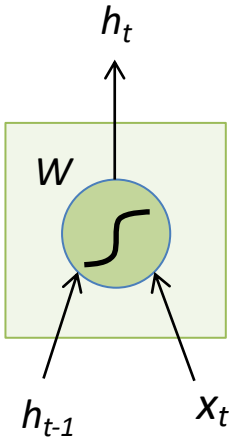
$$h_t = f_W(x_t, h_{t-1})$$

new state | function of W | input at time t | old state

Unrolling the RNN



Vanilla RNN Cell

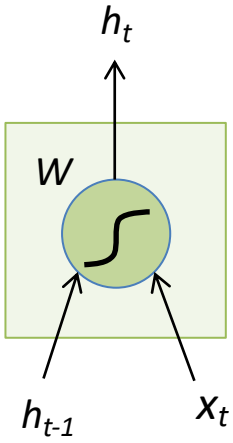


$$h_t = f_W(x_t, h_{t-1})$$

$$= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

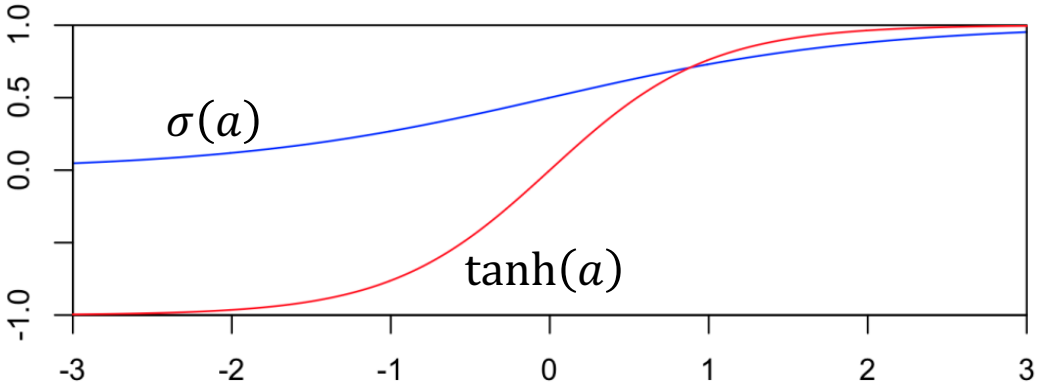
J. Elman, Finding structure in time, Cognitive science 14(2), pp. 179–211, 1990

Vanilla RNN Cell



$$h_t = f_W(x_t, h_{t-1})$$

$$= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

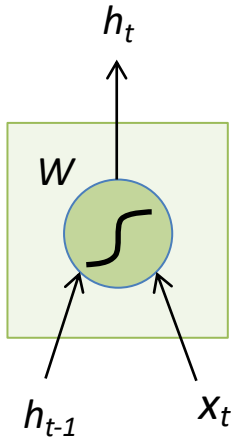


$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

$$= 2\sigma(2a) - 1$$

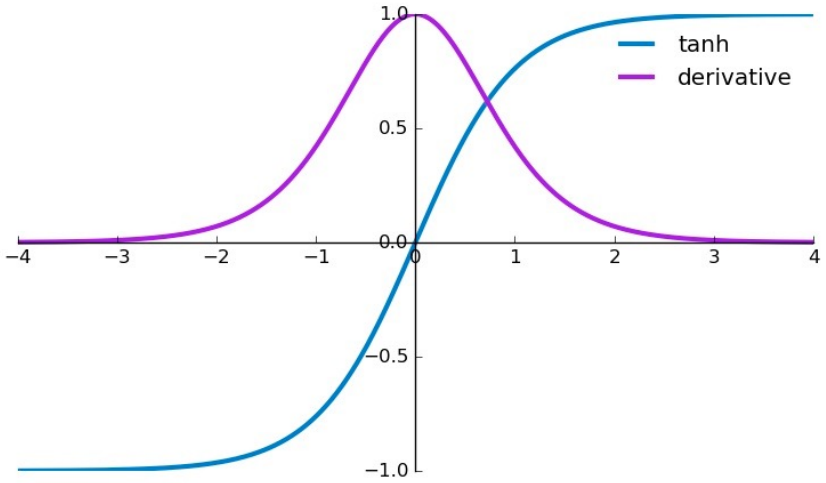
Image source

Vanilla RNN Cell



$$h_t = f_W(x_t, h_{t-1})$$

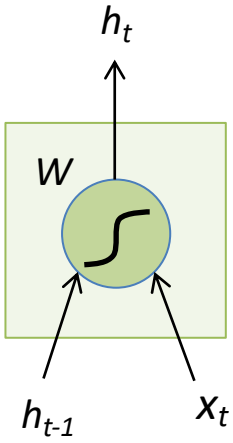
$$= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$



$$\frac{d}{da}\tanh(a) = 1 - \tanh^2(a)$$

Image source

Vanilla RNN Cell



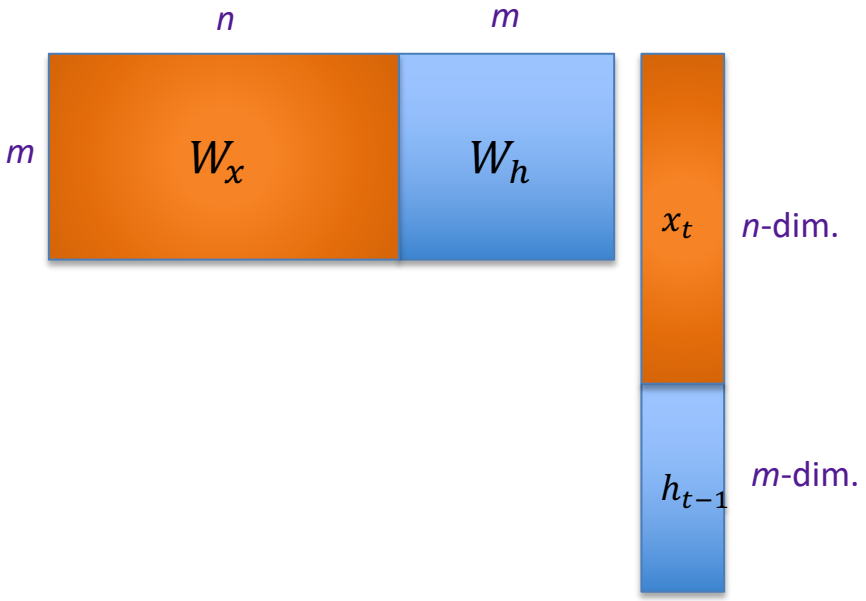
$$h_t = f_W(x_t, h_{t-1})$$

$$= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

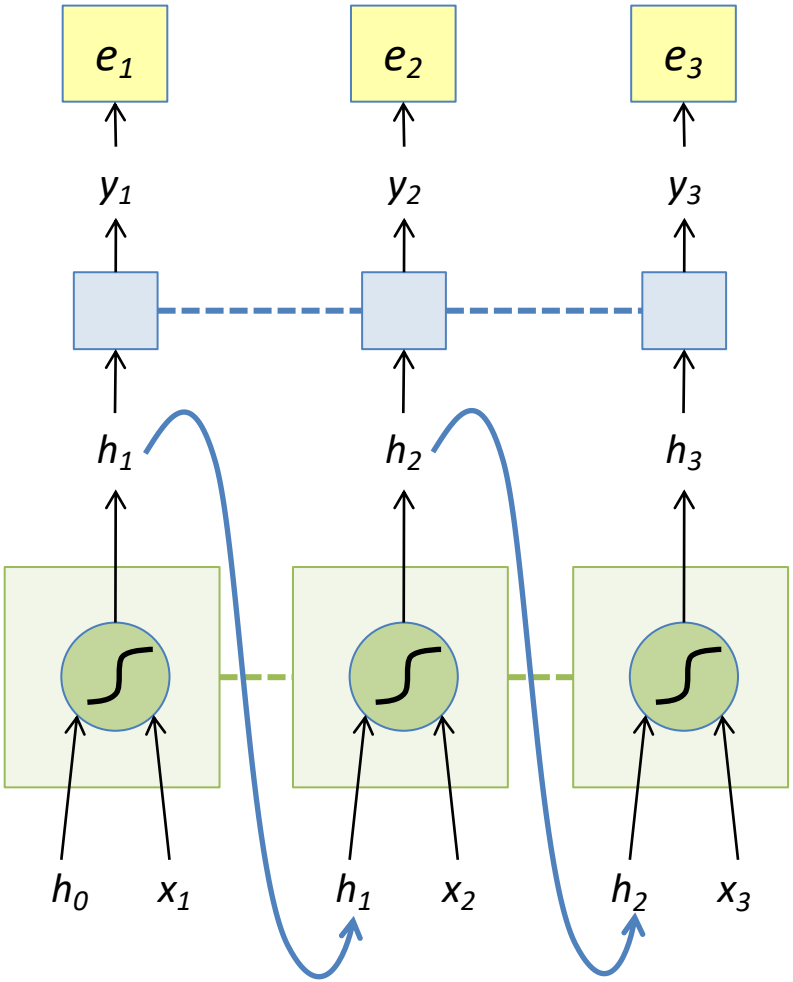
$$= \tanh(W_x x_t + W_h h_{t-1})$$

Q: Why not using ReLU?

- Training can be unstable
- Need good initialization and careful training
  [Le, Jaitly, Hinton]



RNN Forward Pass



$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

------ shared weights

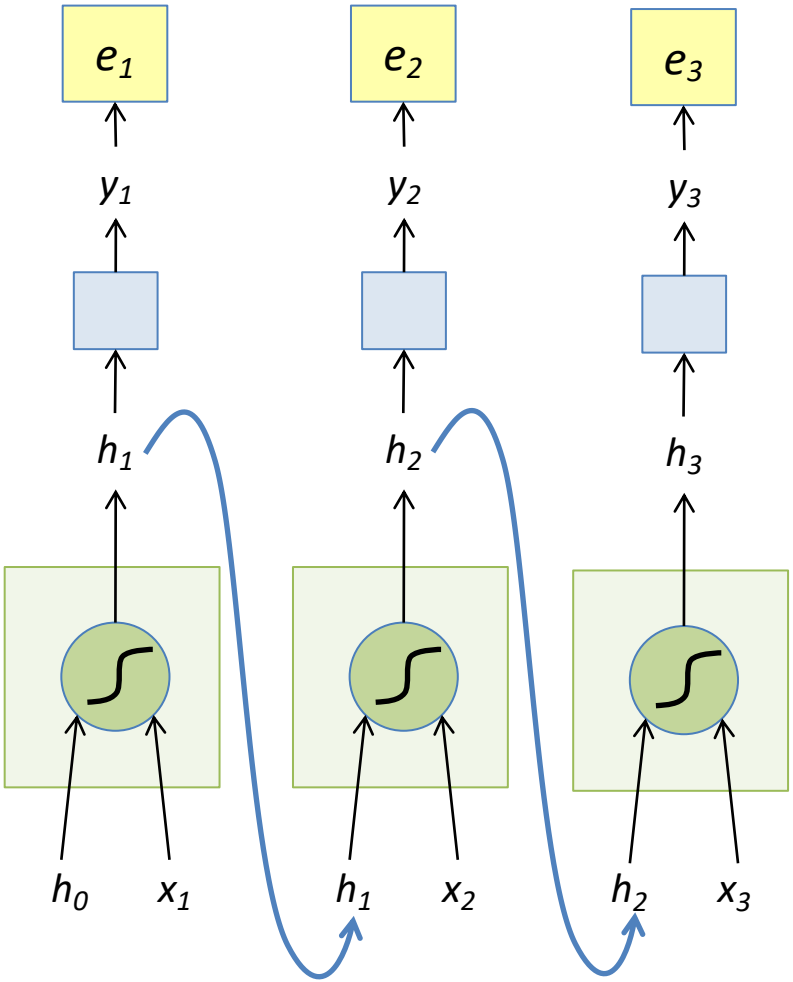
How to train RNNs?

back-propagation!

Backpropagation Through Time (BPTT)

- Most common method used to train RNNs
- The unfolded RNN = one big feed-forward network that accepts the whole time series as input
- Gradients are computed for each copy in the unfolded network, then summed (not averaged!) and applied to the RNN weights

Unfolded RNN Forward Pass

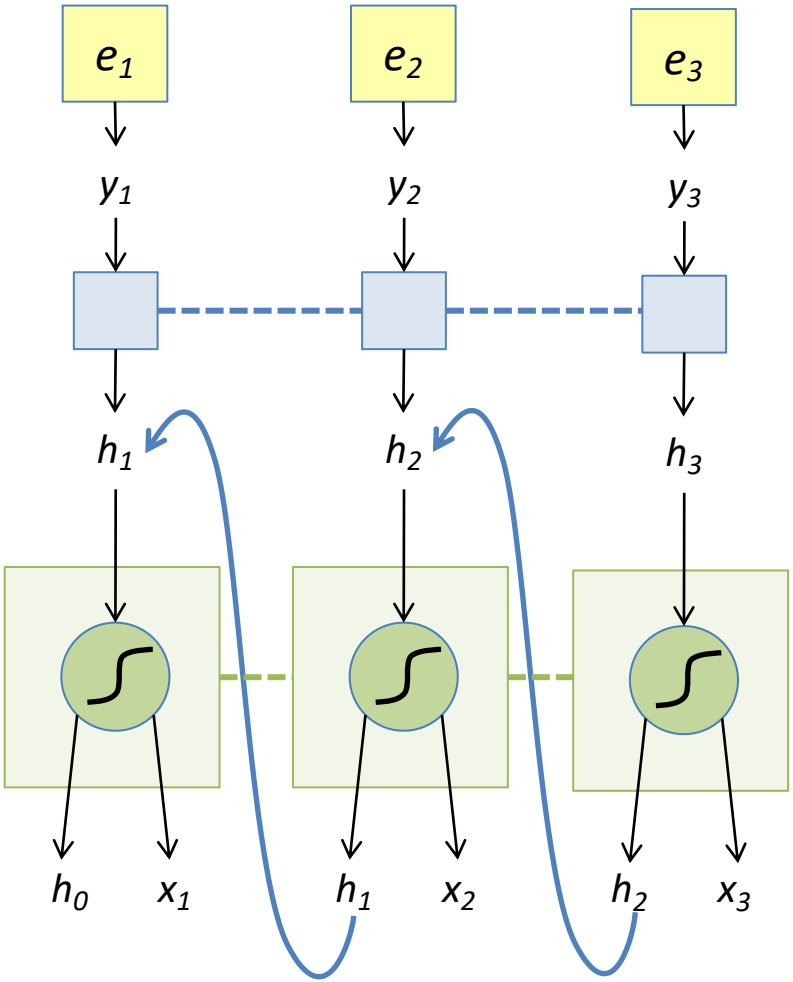


$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

Unfolded RNN Backward Pass



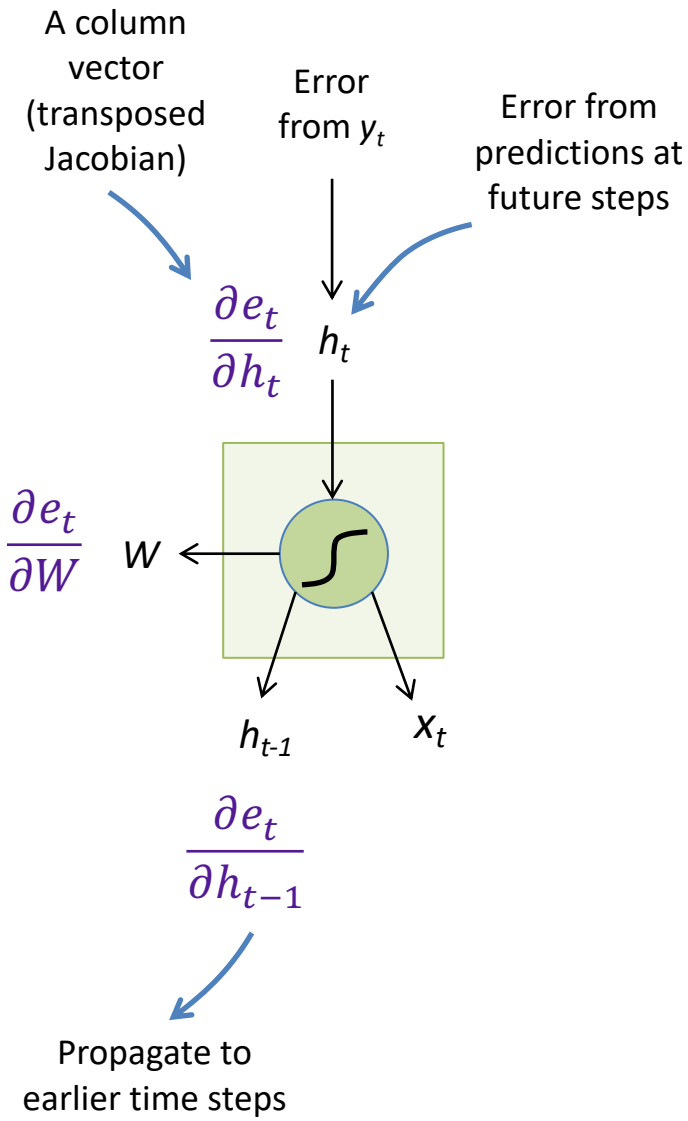
$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

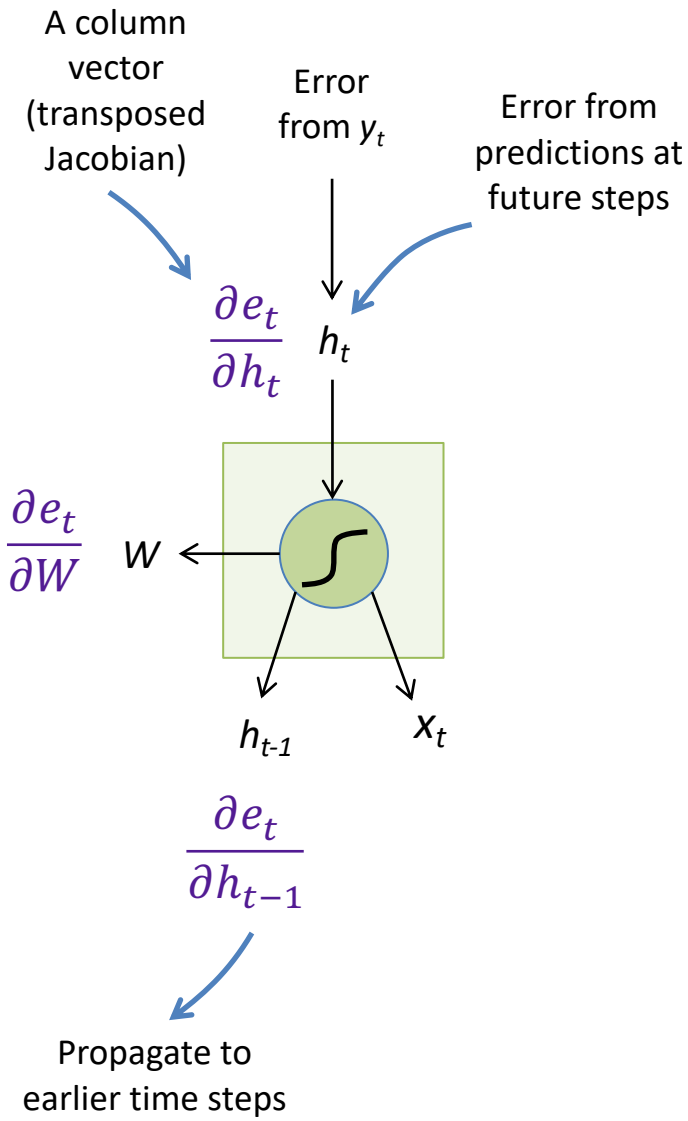
------ Summing gradients (chain rule)

RNN Backward Pass

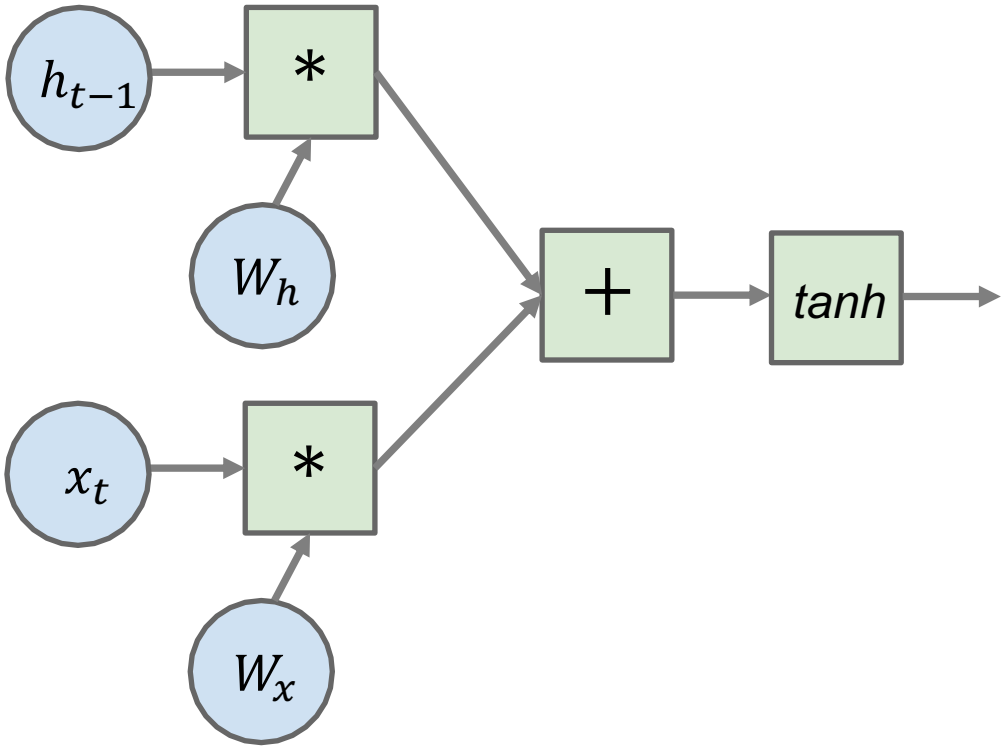


$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

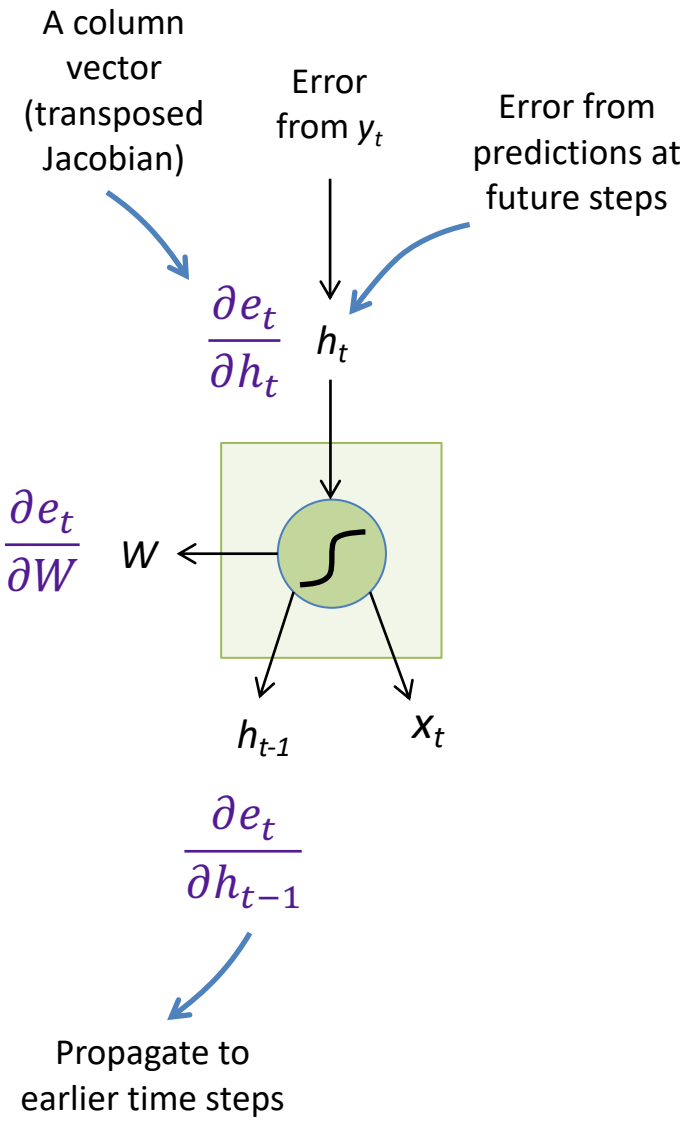
RNN Backward Pass



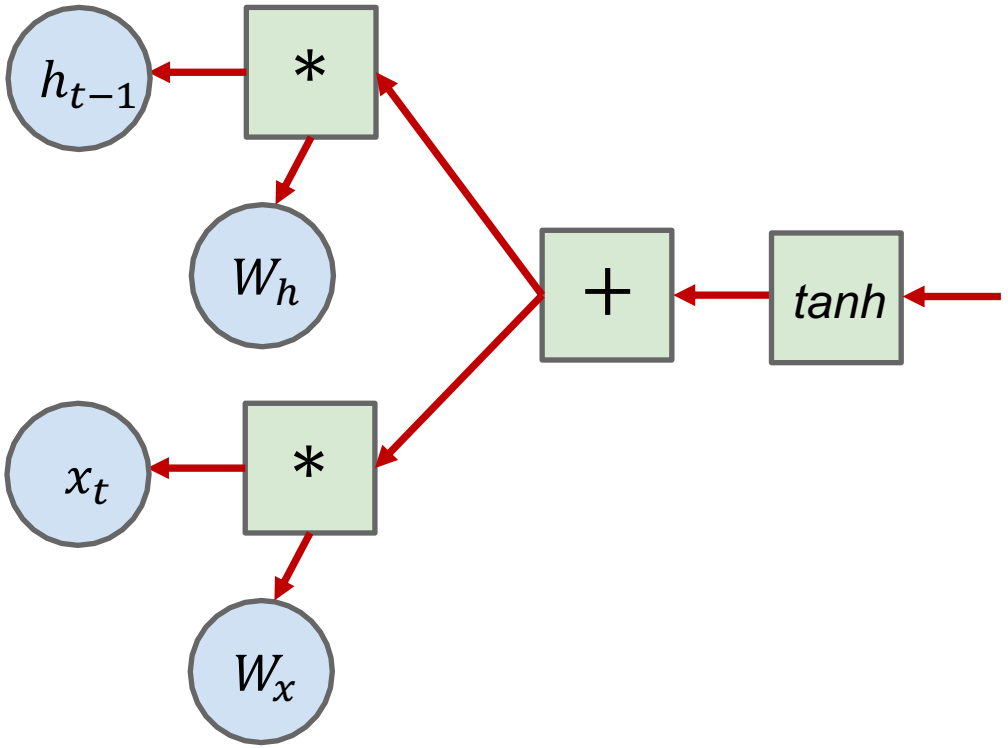
$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$



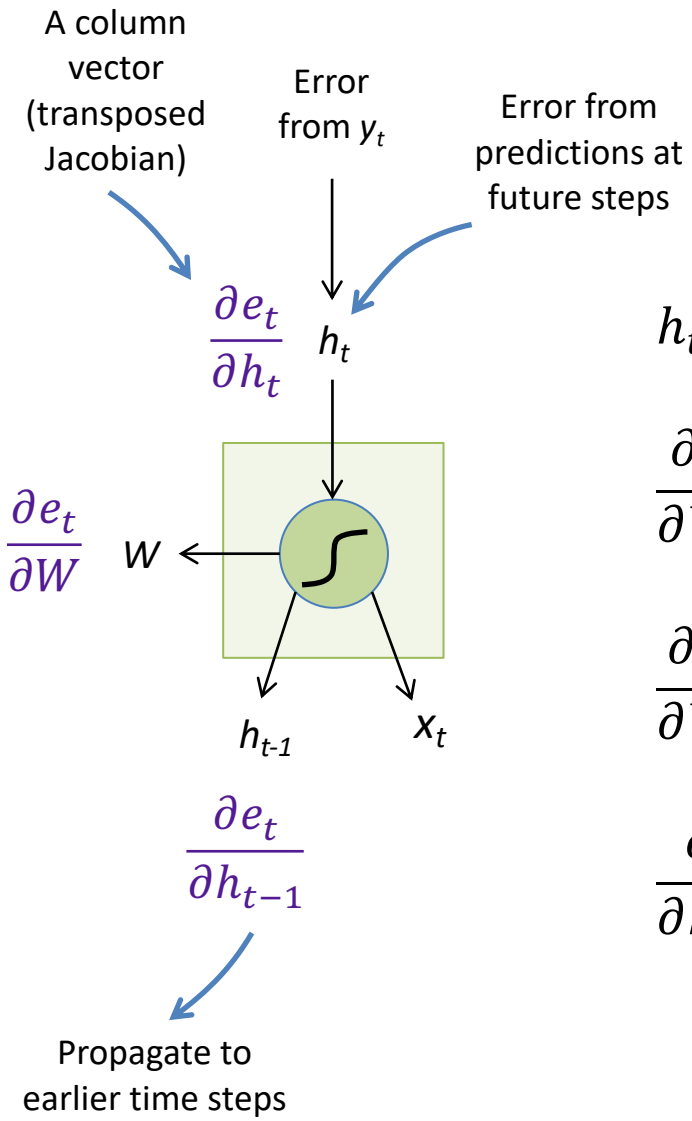
RNN Backward Pass



$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$



RNN Backward Pass



$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

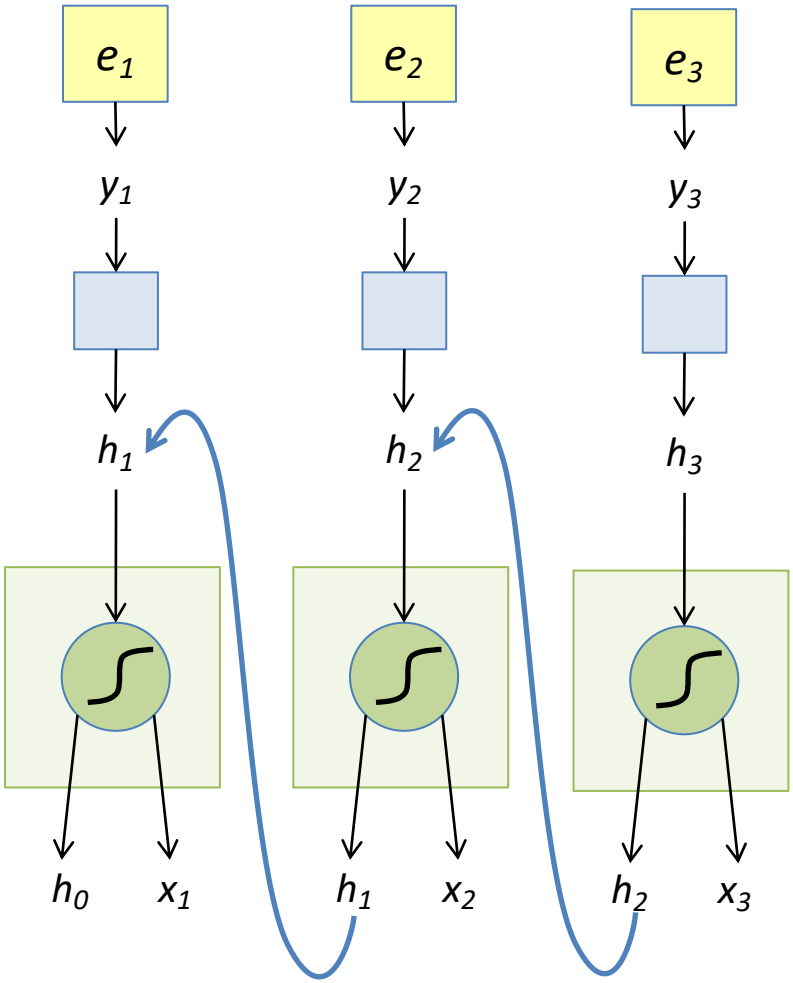
Element-wise multiplication

$$\frac{\partial e_t}{\partial W_h} = \frac{\partial e_t}{\partial h_t} \odot \left(1 - \tanh^2(W_x x_t + W_h h_{t-1})\right) h_{t-1}^T$$

$$\frac{\partial e_t}{\partial W_x} = \frac{\partial e_t}{\partial h_t} \odot \left(1 - \tanh^2(W_x x_t + W_h h_{t-1})\right) x_t^T$$

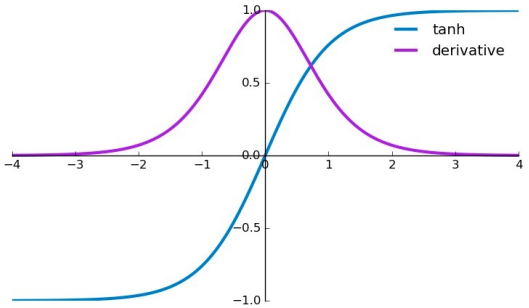
$$\frac{\partial e_t}{\partial h_{t-1}} = \frac{\partial e_t}{\partial h_t} \odot \left(1 - \tanh^2(W_x x_t + W_h h_{t-1})\right) W_h$$

RNN Backward Pass



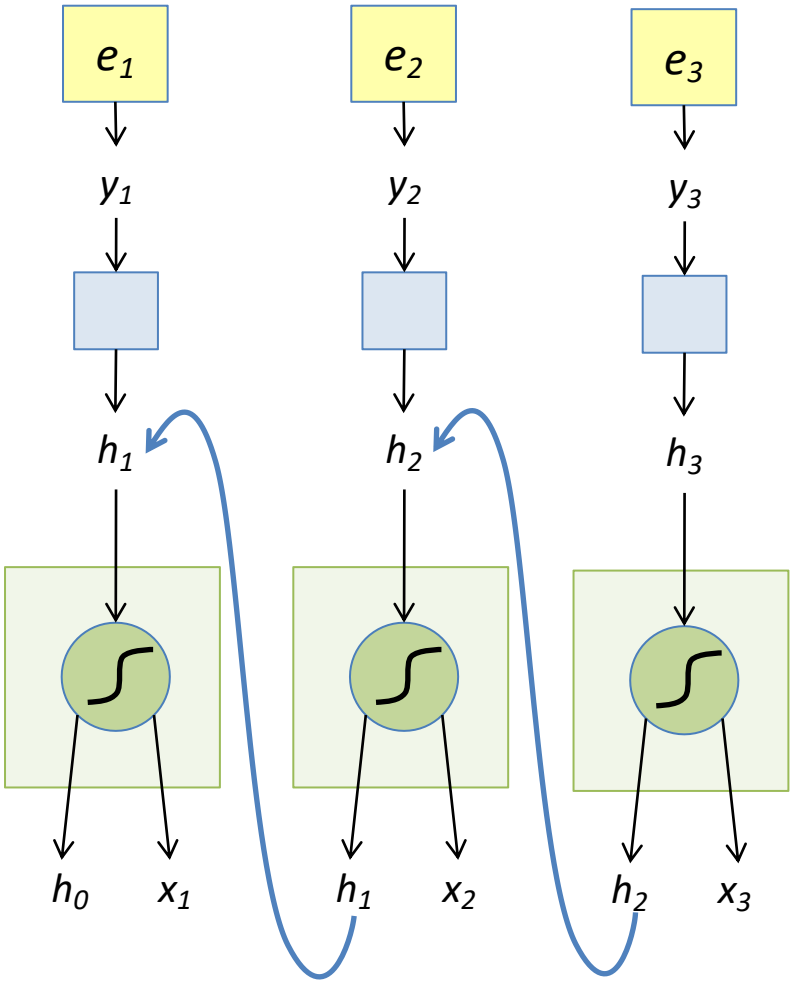
$$\frac{\partial e_t}{\partial h_{t-1}} = W_h^T\left(1 - \tanh^2(W_x x_t + W_h h_{t-1})\right) \odot \frac{\partial e}{\partial h_t}$$

Large tanh activations will give small gradients



Consider $\frac{\partial e_n}{\partial h_k}$ for $k \ll n$

RNN Backward Pass



$$\frac{\partial e_t}{\partial h_{t-1}} = W_h^T\left(1 - \tanh^2(W_x x_t + W_h h_{t-1})\right) \odot \frac{\partial e}{\partial h_t}$$

Gradients will vanish if largest singular value of W_h is less than 1

Unrolling can yield a *very deep feed-forward network*!

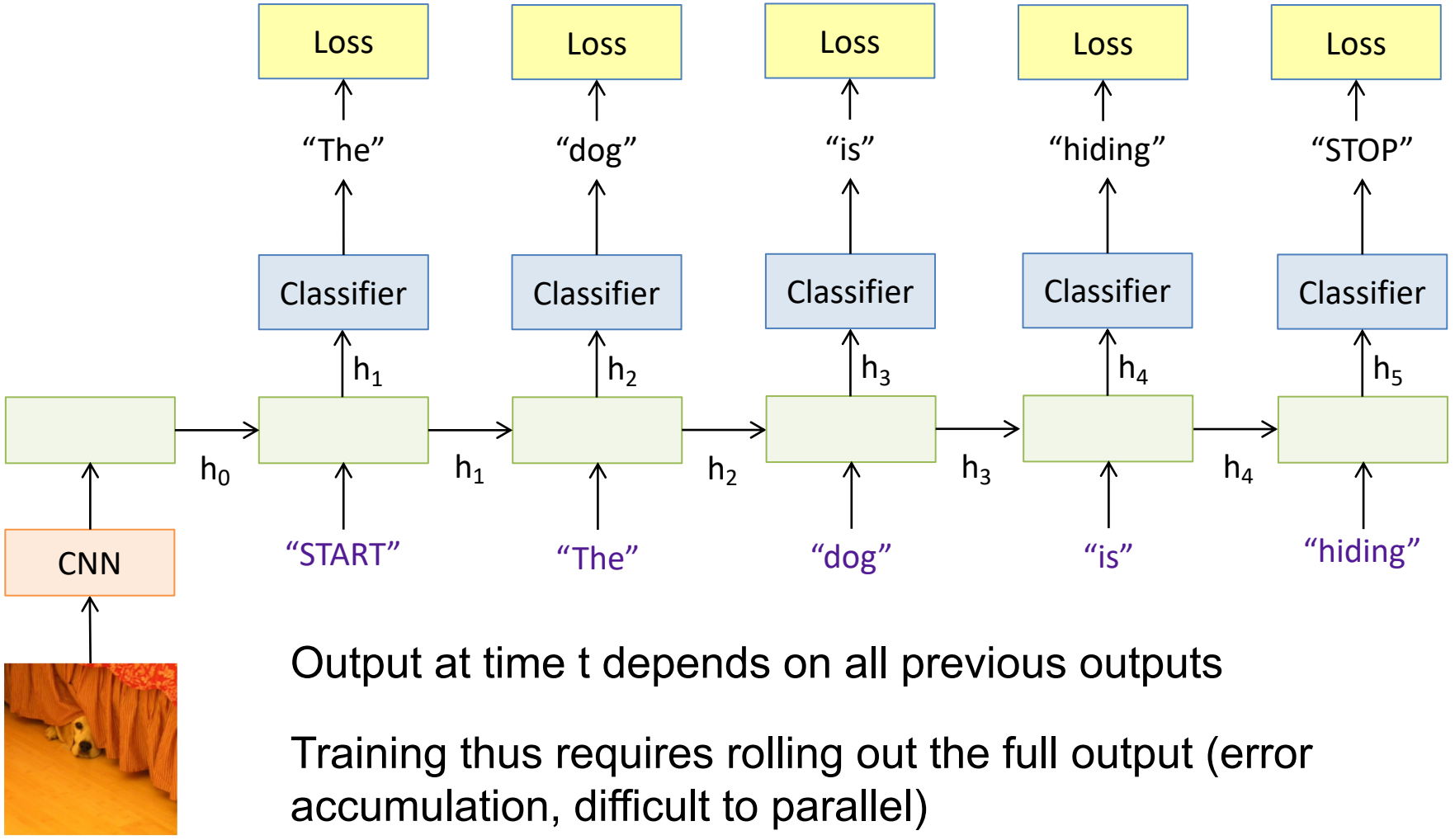
Consider $\frac{\partial e_n}{\partial h_k}$ for $k \ll n$

Backpropagation Through Time (BPTT)

- Most common method used to train RNNs
- The unfolded RNN = one big feed-forward network that accepts the whole time series as input
- Gradients are computed for each copy in the unfolded network, then summed (or averaged) and applied to the RNN weights
- In practice, *truncated* BPTT is used: run the RNN forward k_1 time steps, propagate backward k_2 time steps
  - $k_1 = k_2$ *for most of the cases*

https://machinelearningmastery.com/gentle-introduction-backpropagation-time/
http://www.cs.utoronto.ca/~ilya/pubs/ilya_sutskever_phd_thesis.pdf

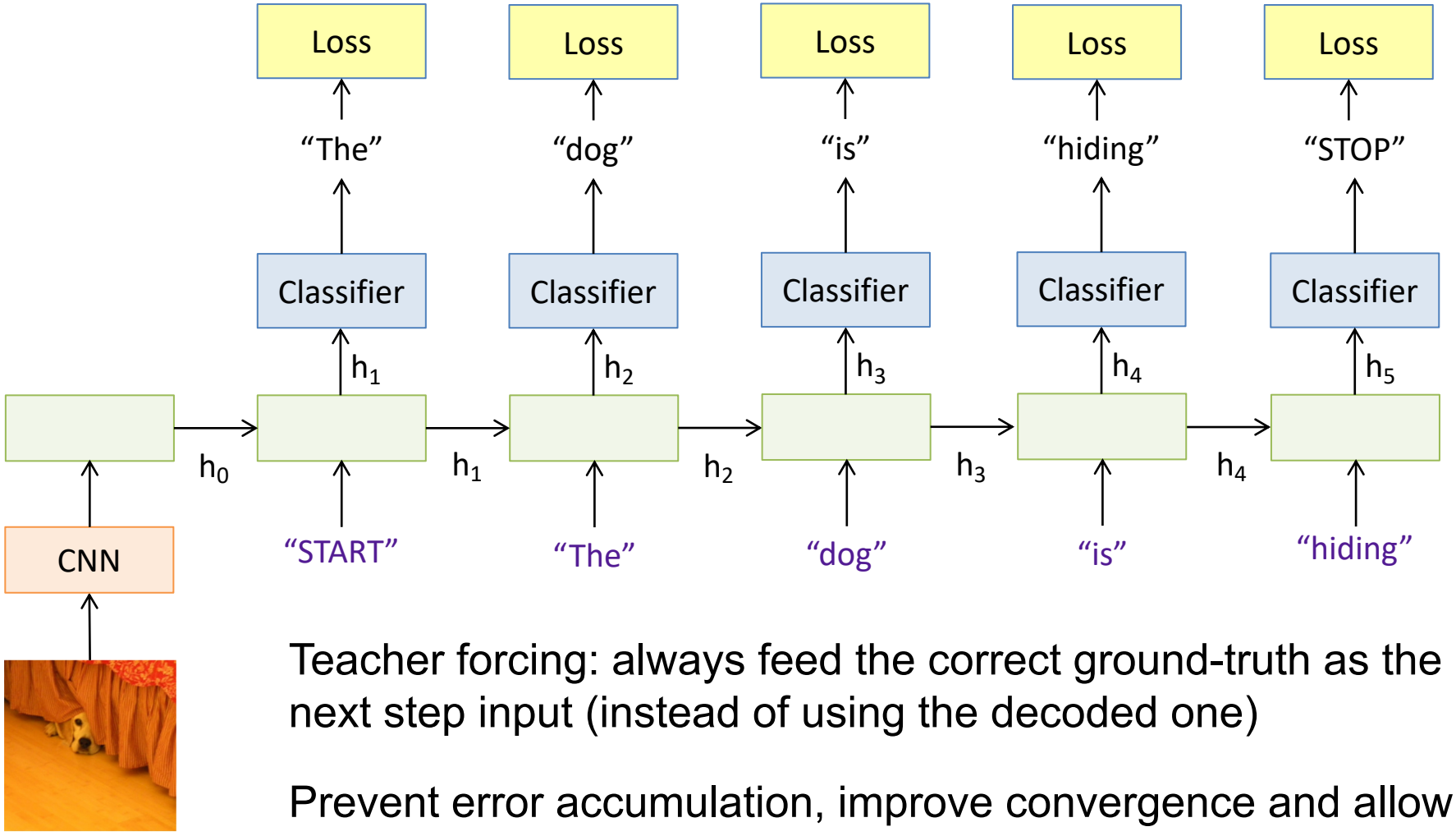
RNN Training: Practical Considerations



Output at time t depends on all previous outputs

Training thus requires rolling out the full output (error accumulation, difficult to parallel)

RNN Training: Practical Considerations

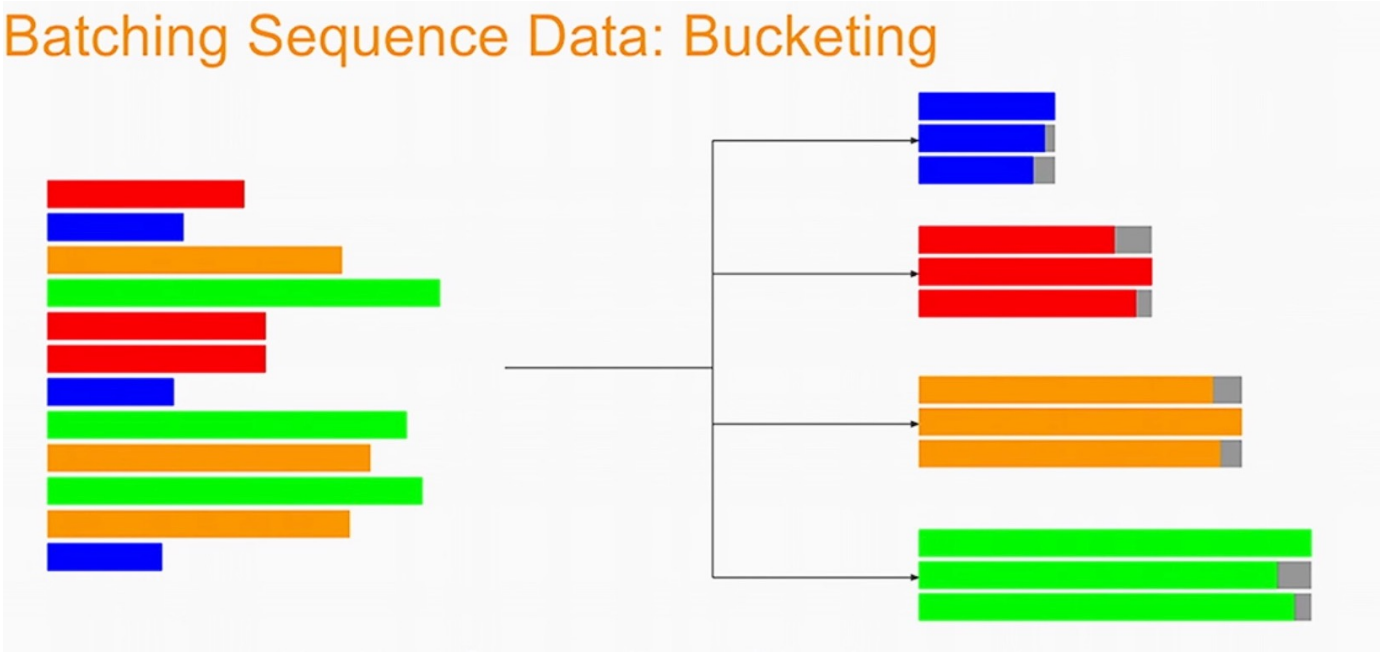


Teacher forcing: always feed the correct ground-truth as the next step input (instead of using the decoded one)

Prevent error accumulation, improve convergence and allow parallelism (more later …)

RNN Training: Practical Considerations

- Output sequence may have very different length
- Bucket the samples based on their output length (assuming using teacher forcing), and only sample from one bucket for each mini-batch



Source: https://medium.com/@rashmi.margani/how-to-speed-up-the-training-of-the-sequence-model-using-bucketing-techniques-9e302b0fd976

Memory in RNNs

Recurrence:

$$h_t = f_W\,(x_t, h_{t-1})$$

current state | function of W | input at time t | old state

Memory:

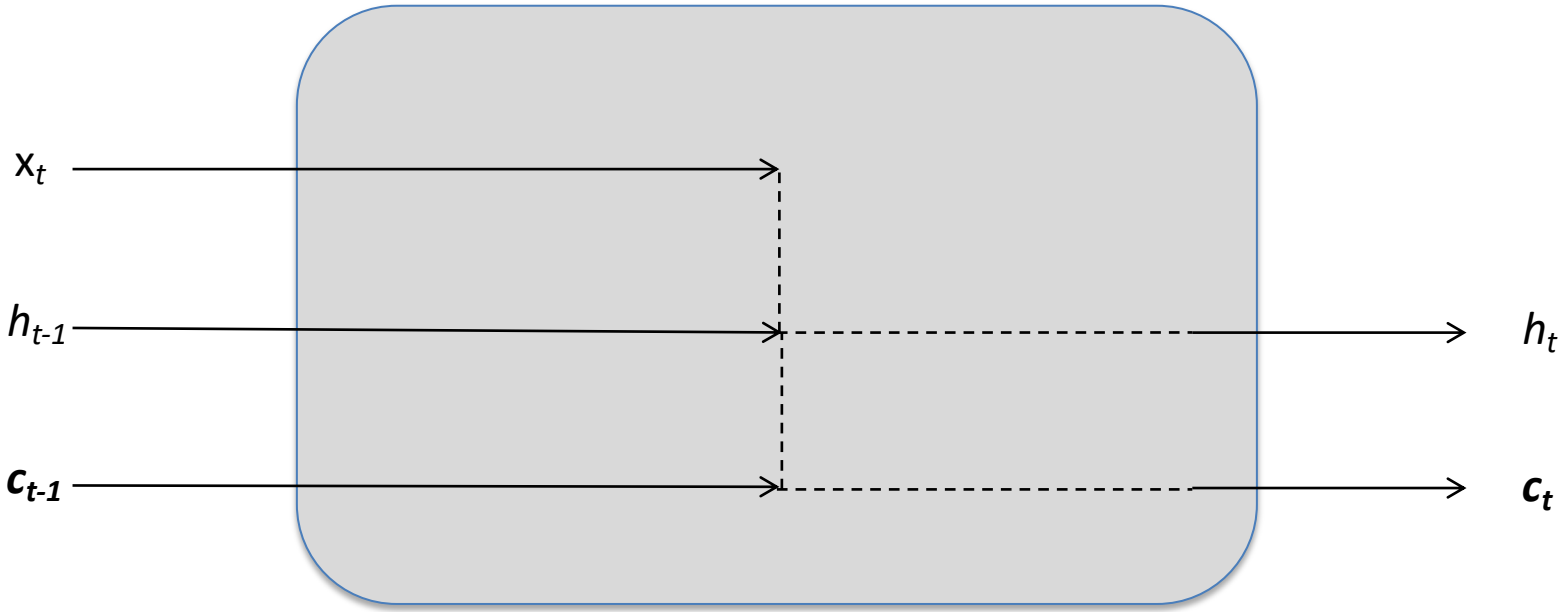
$$h_t = \Phi\,(x_{1:t}, h_0)$$

current state | input up to time t | initial state

- RNNs can be viewed as an aggressive compression
- Input sequence of any length is summarized using a hidden state (a fixed length vector)
- Gradient vanishing: gradients only propagate to a few recent steps (i.e., only the short-term memory is learned at training)

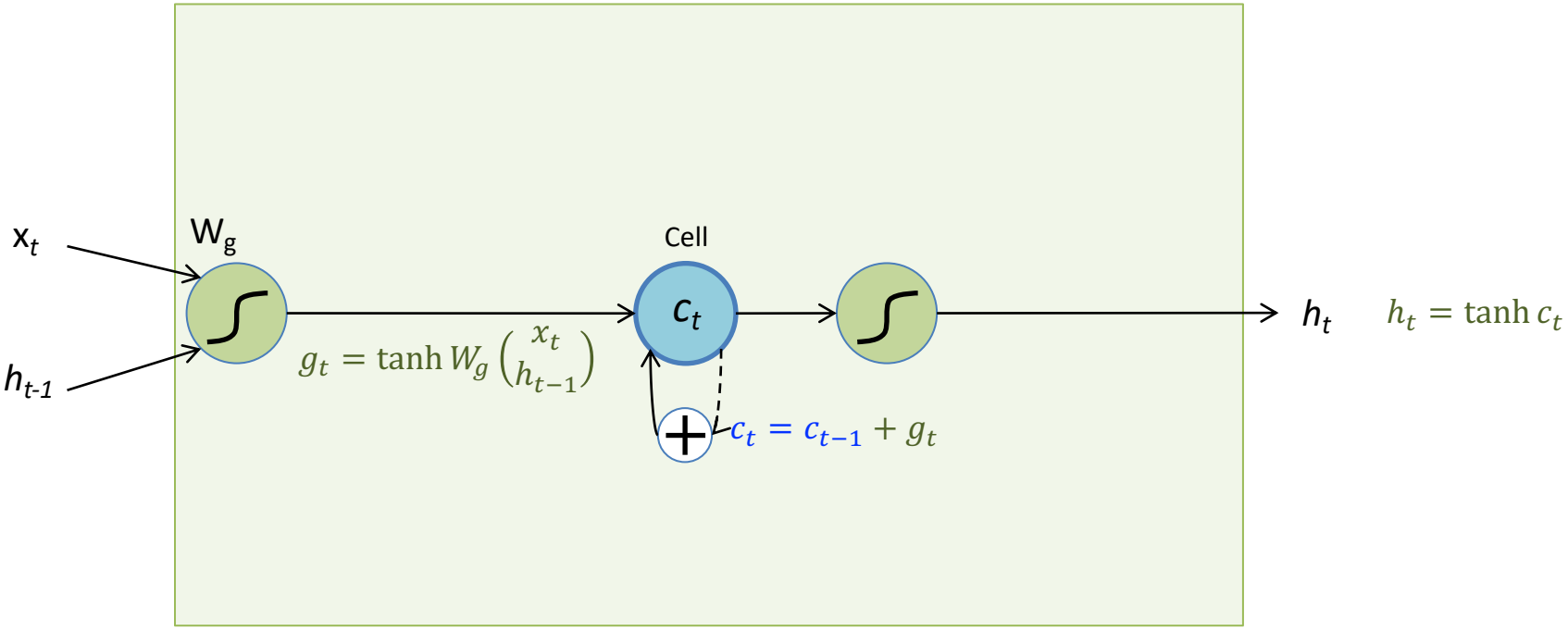
Long Short-Term Memory (LSTM)

- Add a *memory cell* that is not subject to matrix multiplication or squishing, thereby avoiding gradient decay



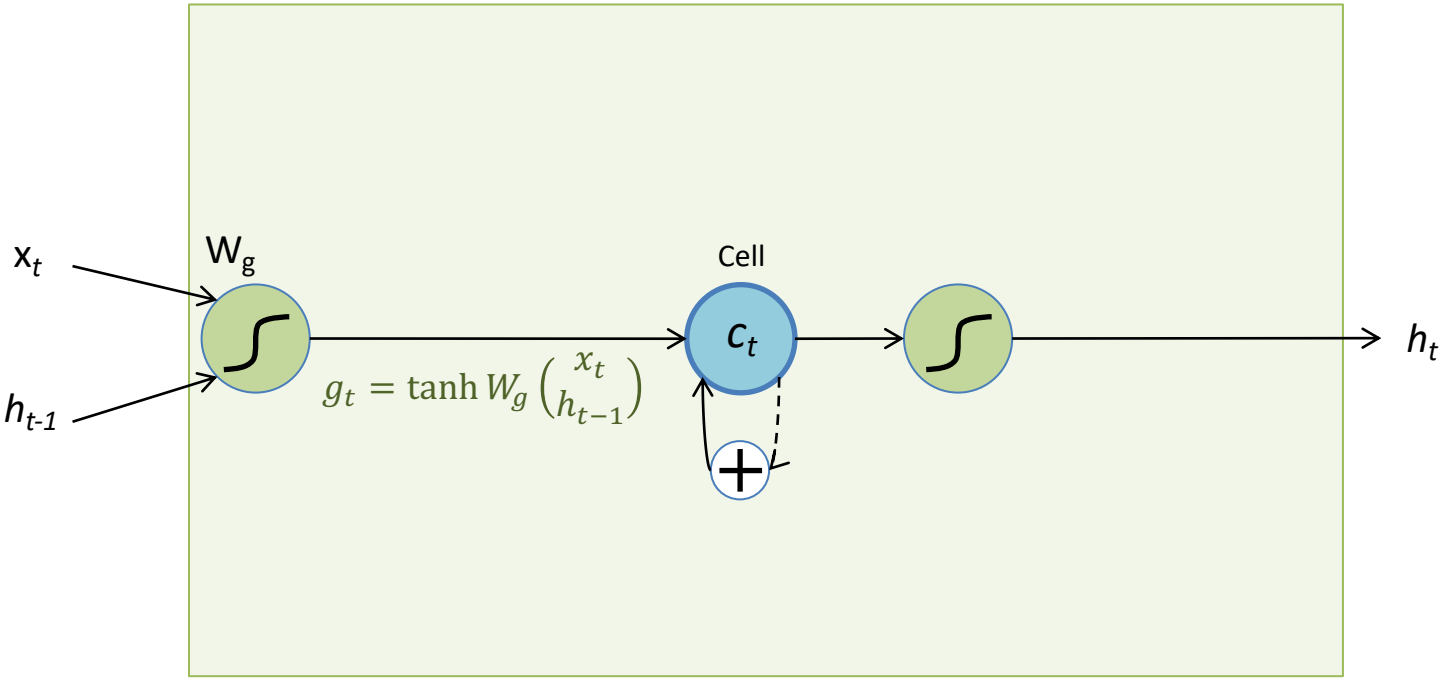
S. Hochreiter and J. Schmidhuber, Long short-term memory, Neural Computation 9 (8), pp. 1735–1780, 1997

The LSTM Cell

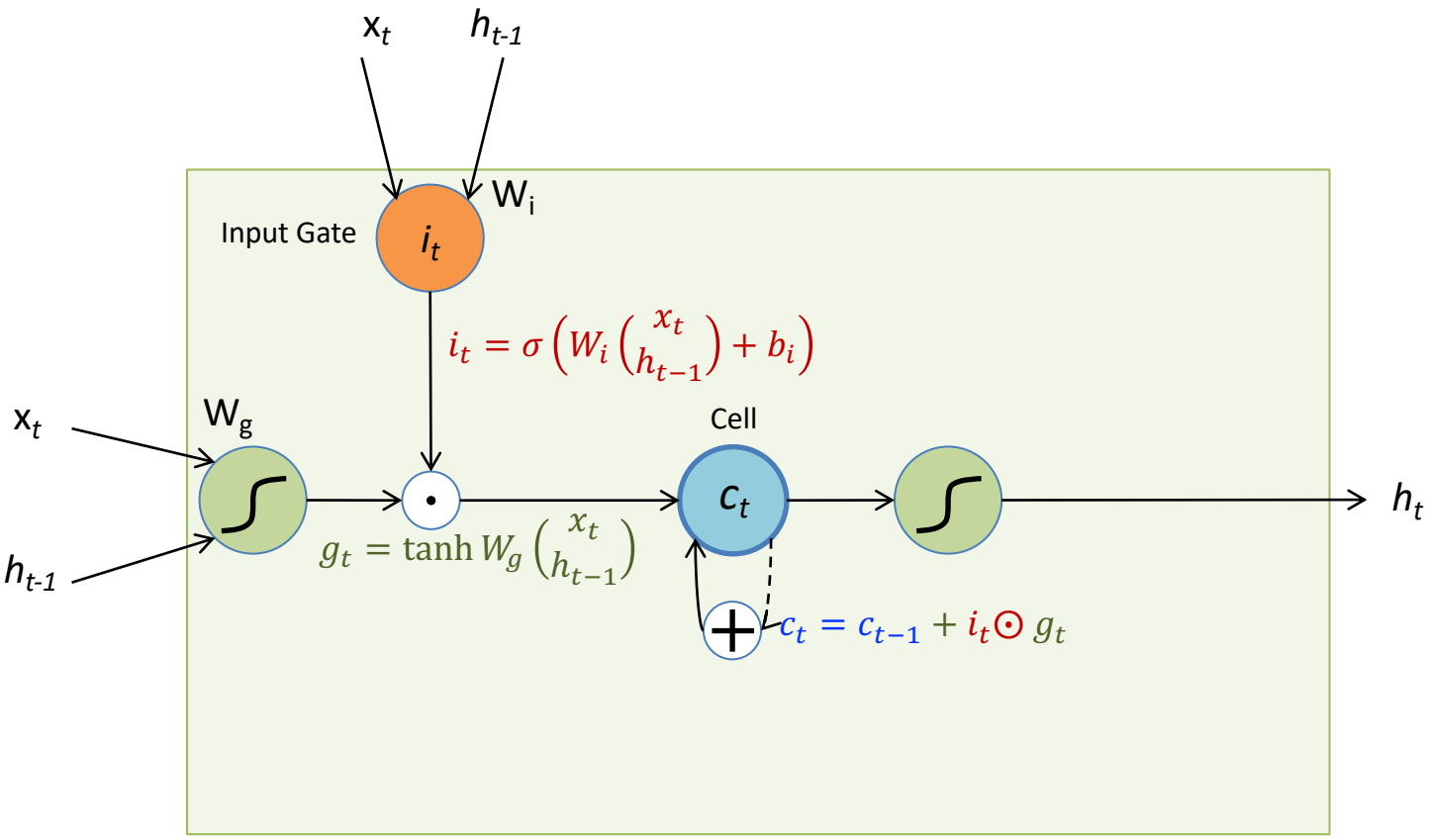


* Dashed line indicates time-lag

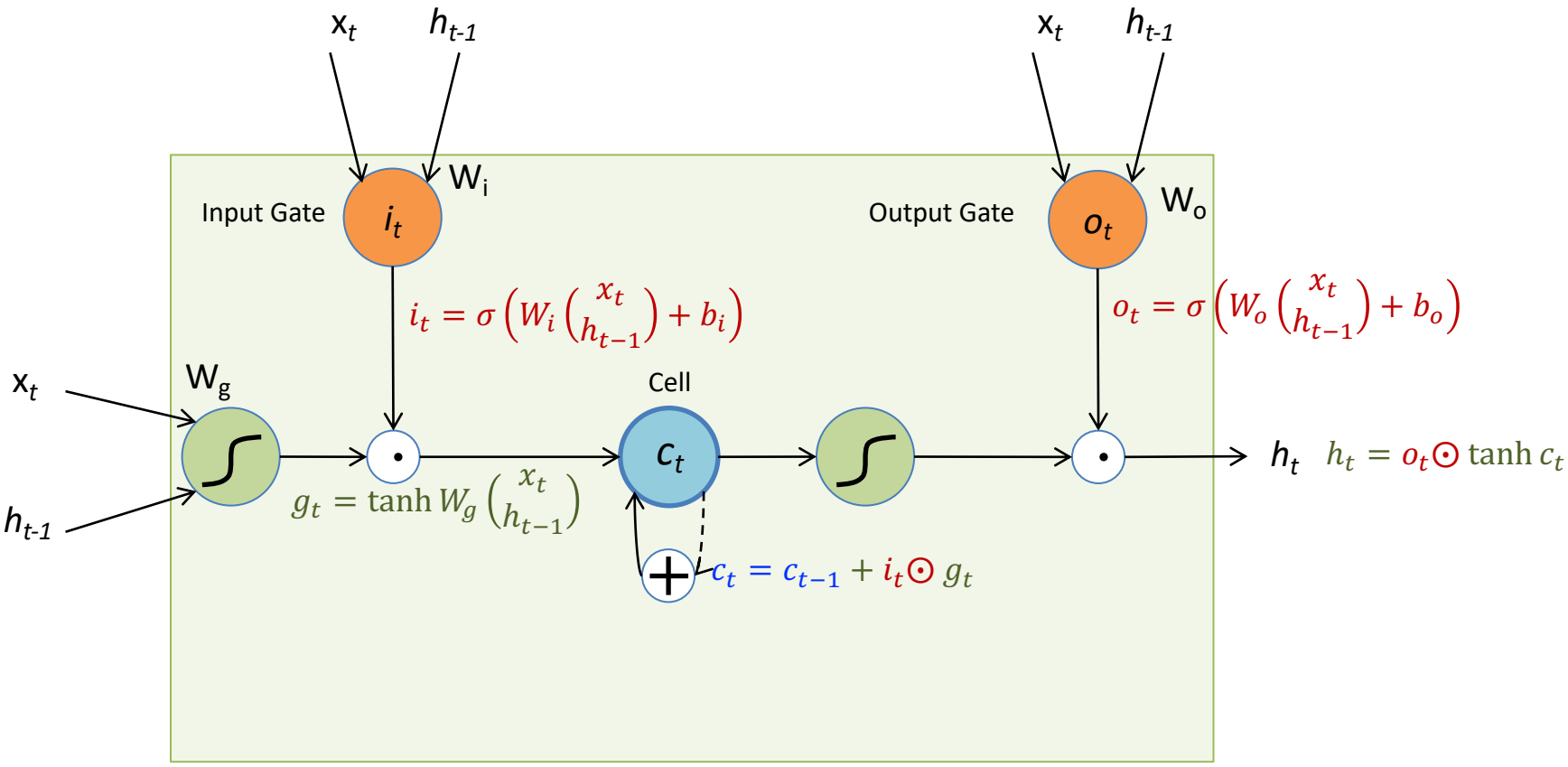
The LSTM Cell



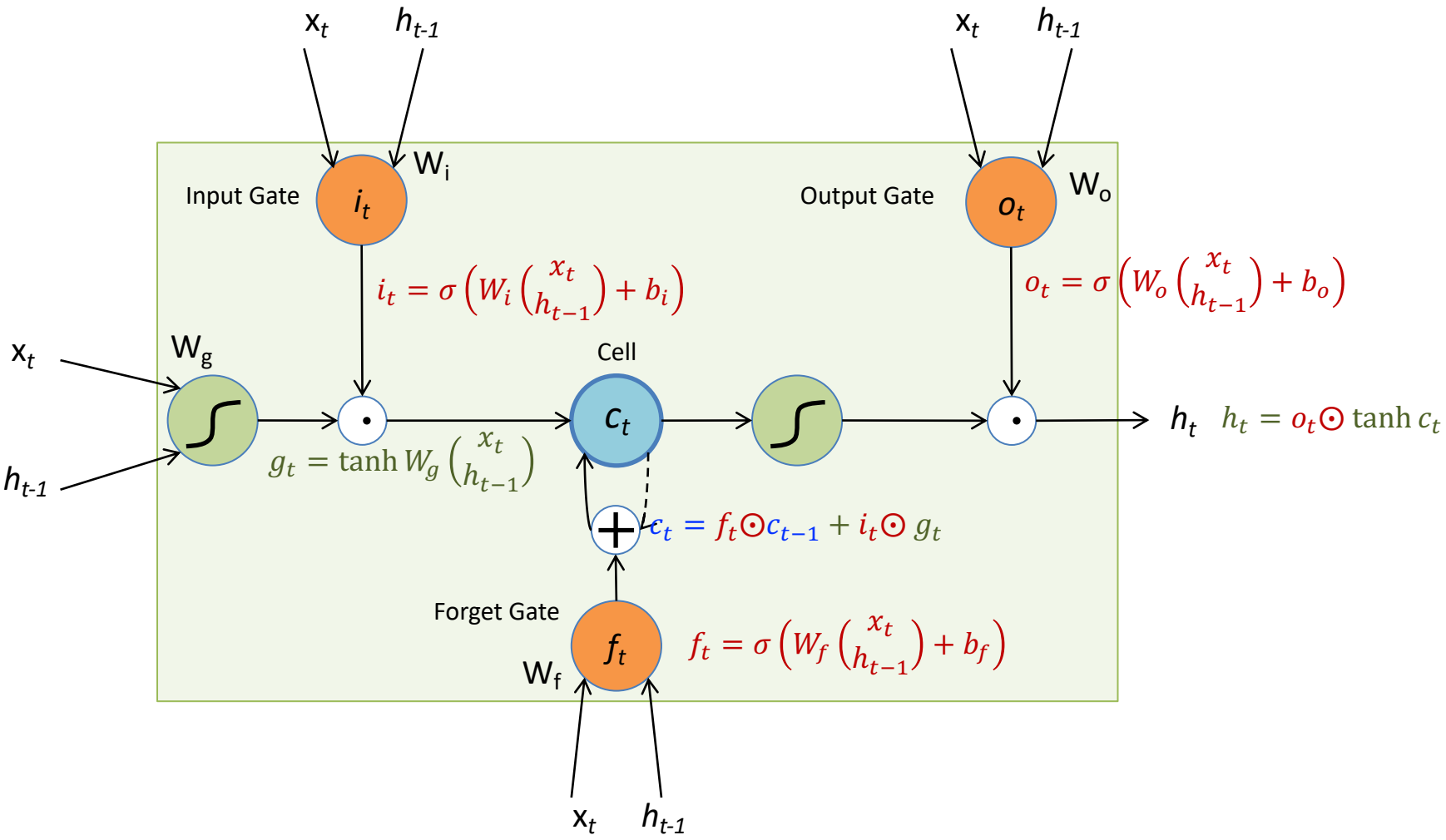
The LSTM Cell



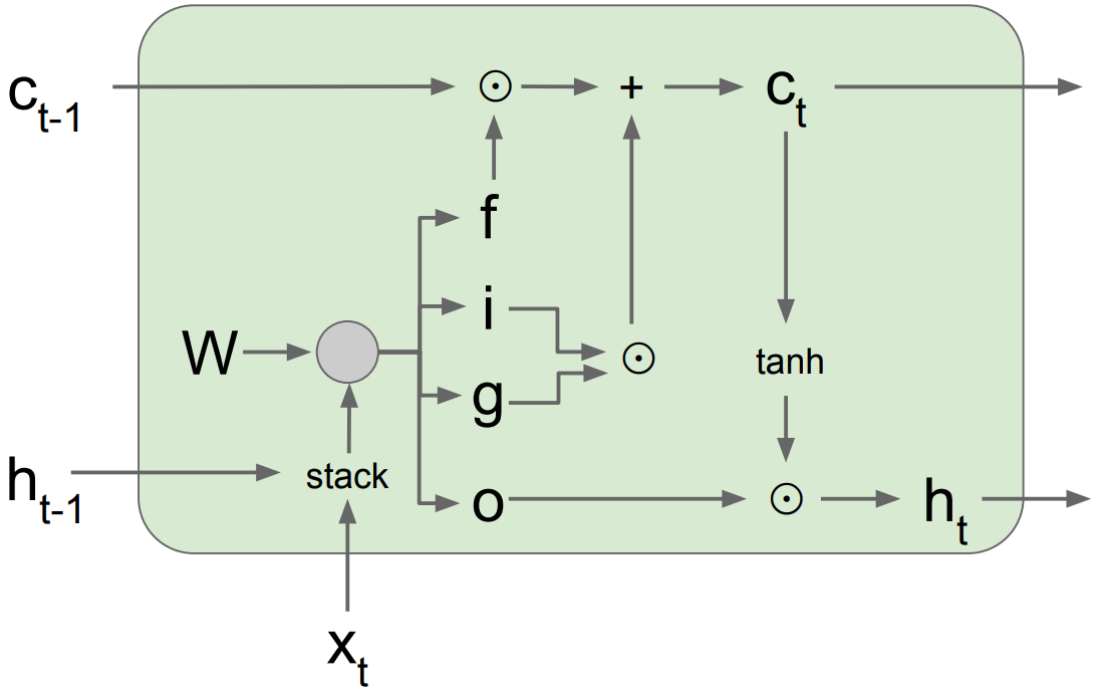
The LSTM Cell



The LSTM Cell



LSTM Forward Pass Summary



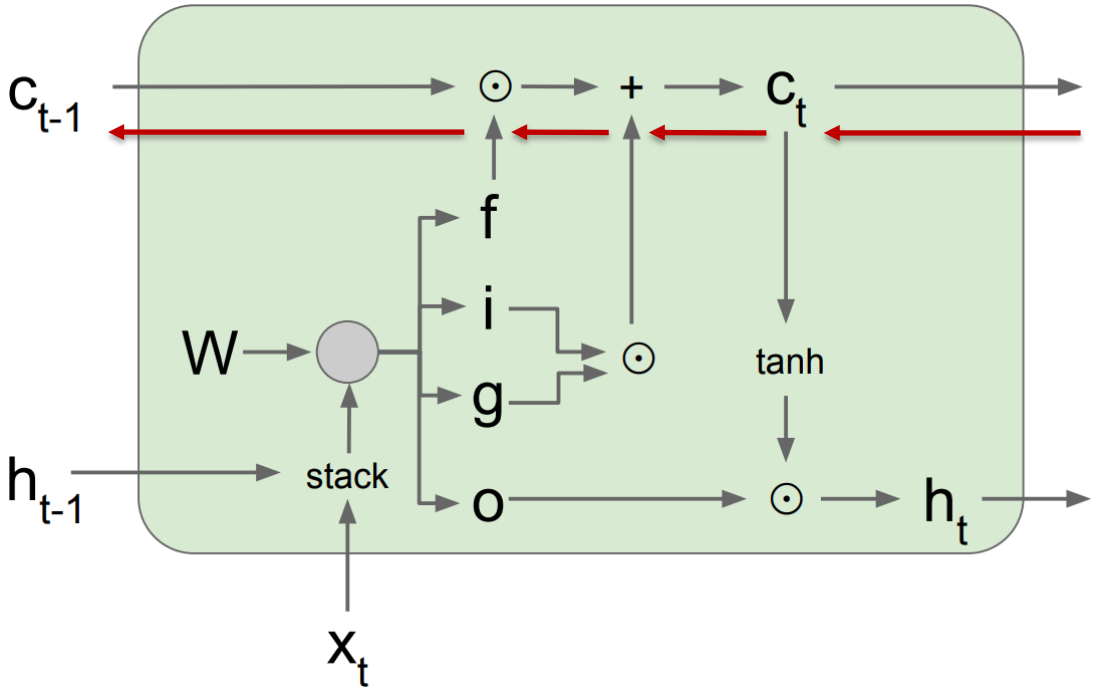
$$\begin{pmatrix} g_t \\ i_t \\ f_t \\ o_t \end{pmatrix} = \begin{pmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{pmatrix} \begin{pmatrix} W_g \\ W_i \\ W_f \\ W_o \end{pmatrix} \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh c_t$$

Figure source

LSTM Backward Pass



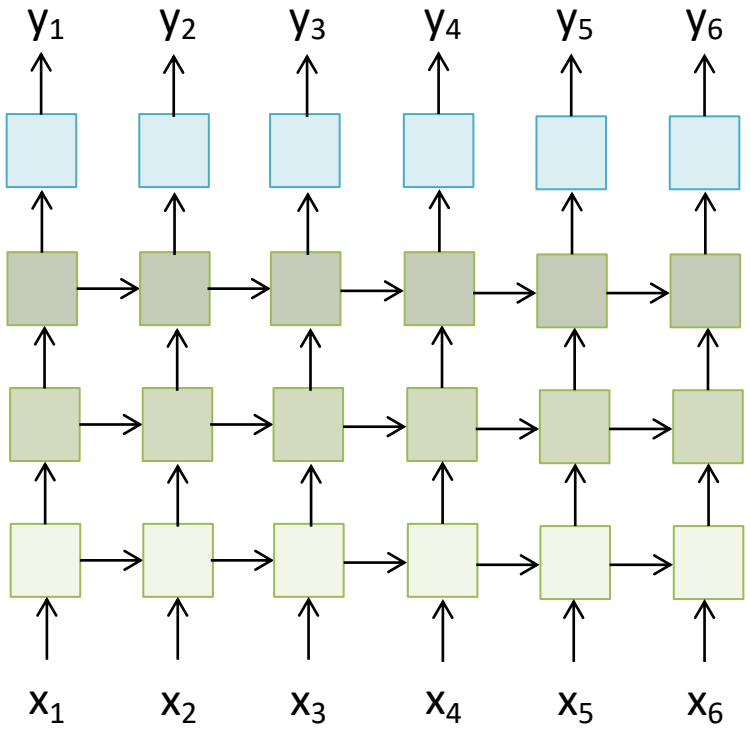
Gradient flow from c_t to c_{t-1} only involves back-propagating through addition and elementwise multiplication, not matrix multiplication or tanh

For complete details: Illustrated LSTM Forward and Backward Pass

Figure source

Multi-layer RNNs / LSTMs

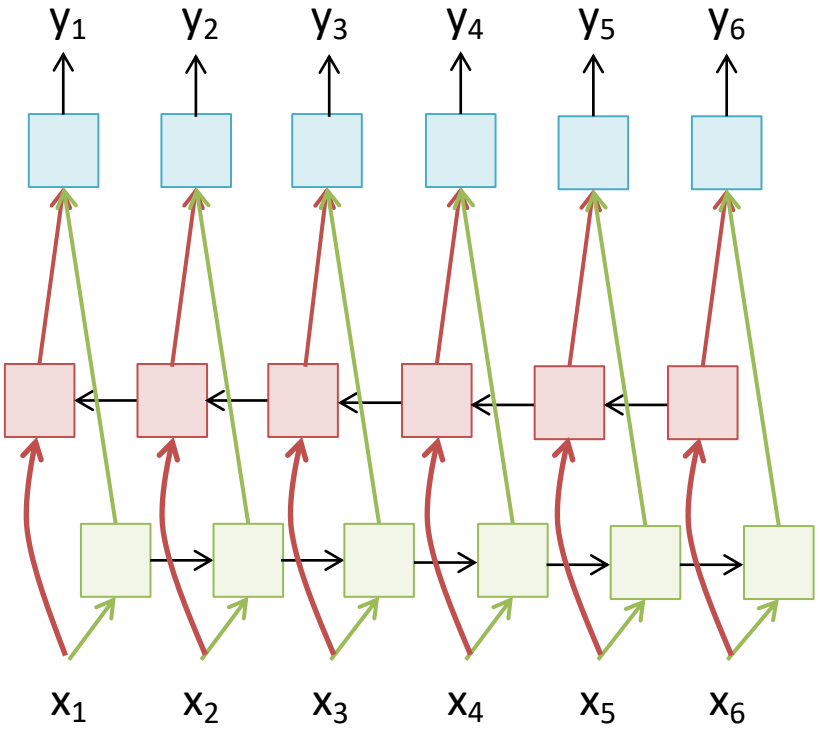
- We can of course design RNNs with multiple hidden layers



- Skip connections across layers, across time, …

Bi-directional RNNs / LSTMs

- RNNs can process the input sequence in forward and in the reverse direction



- Was popular in speech recognition / NLP

RNNs?

- Training can be time / memory consuming (unrolling produces huge feedforward models)
  - *truncated* BPTT
  - teacher forcing & bucketing

- Gradient vanishing (largest singular value of W_h<1)
  - Adding “skip connections” during backpropagation -> LSTMs
  - Include the full history in the recurrence -> Transformers

- Many of the practices are broadly applicable (e.g., for Transformers)

To Be Continued